

SOLUTION REQUIREMENTS

Date: 11 July 2026

Team ID: XXXXXX

Project Name: EduGenie – Google Gemini Powered Learning Assistant Maximum

Marks: 4 Marks

Define Functional and Non-Functional Requirements

This document defines the functional and non-functional requirements for EduGenie – Google Gemini Powered Learning Assistant. Functional requirements describe the features and services provided by the system, while non-functional requirements specify the quality, performance, security, reliability, and usability expectations.

Step 1: Functional Requirements (FR)

S.No	Requirement		Priority
	Category	Requirement Description	
1		Secure login and user session management for students and teachers.	User Authentication High
2	AI Question Answering	Answer academic questions using Google Gemini 1.5 Pro and LaMini-Flan-T5 models.	High
3		Generate clear and easy-to-understand explanations for academic concepts.	Concept Explanation High
4		Automatically generate quizzes and multiple-choice	Quiz Generation High

		questions based on the selected topic. Summarize lengthy educational content into concise	
5			Text Summarization High
		notes.	
6	Personalized	Recommend study plans and learning resources based on	High
	Learning Path	student interests and learning progress. Integrate Google Gemini API with the FastAPI backend	
7			API Integration High
		for AI-powered educational services.	
8	Dashboard & History	Display previous learning activities, generated content, and recommendations.	Medium

Step 2: Non-Functional Requirements (NFR)

		Target Metric / Acceptance Criteria
S.No	NFR Category	Requirement Description
1	Performance & Speed	Generate AI responses quickly for all educational tasks. Response within 5 seconds (excluding AI model latency).
2		Support multiple users accessing the application simultaneously. Support 100+ concurrent Scalability users.
3	Security & Data Privacy	Protect user information and HTTPS communication and learning history using secure encrypted data storage.

		communication.	
4	Reliability & Availability	Ensure continuous availability of the application.	99% system uptime.
5	Usability & Accessibility	Provide a responsive and user-friendly interface for students and teachers.	Compatible with desktop, tablet, and mobile devices.
6	Maintainability & Portability	Design the application with modular Python code and reusable components for easy maintenance and deployment.	Modular architecture with reusable FastAPI modules and easy deployment across platforms.